

Лабораторная работа №2

Цель работы: создание веб-приложения с использованием фреймворка Flask с интеграцией предварительно обученной модели искусственной нейронной сети для классификации изображений

Ход работы: Создайте проект с произвольным названием.

Создайте виртуальное окружение для проекта которое назовите `venv`. Для этого выполните команду:

```
python -m venv venv
```

Для активации виртуального окружения выполните команду:

```
source venv/bin/activate (Linux/macOS)
```

```
venv\Scripts\activate (Windows)
```

В случае использования **PyCharm IDE** данные команды выполнять *не требуется* поскольку виртуальное окружение будет создано и активировано автоматически.

В проекте создайте python файл и назовите его `app.py`

Для начала работы с Flask в терминале выполните команду для установки фреймворка

```
pip install flask
```

После чего установщик пакетов должен установить фреймворк.

Установите фреймворк Tensorflow (обязательно той же версии, что и в среде, где выполнялось обучение модели), в данном случае версия будет 2.19.0

```
pip install tensorflow==2.19.0
```

Установите библиотеку PIL, она позже потребуется для работы с изображениями. Ее можно установить командой:

```
pip install Pillow
```

Выполните команду

```
pip freeze > requirements.txt
```

для сохранения зависимостей проекта.

В данной лабораторной работе необходимо создать домашнюю страницу приложения для возможности загрузки изображения для его классификации. Реализация загрузки изображения будет сделана позже. На данном этапе требуется создать соответствующий шаблон.

Все шаблоны (html страницы), которые будет возвращать сервер по соответствующим запросам должны находиться в пакете `templates`. Создайте его в проекте как показано на рисунке 1. В директории `templates` создайте два шаблона `index.html`, для домашней страницы в которой пользователь сможет затем имея соответствующий интерфейс загружать изображения для анализа. Шаблон `temp1` понадобится позже.

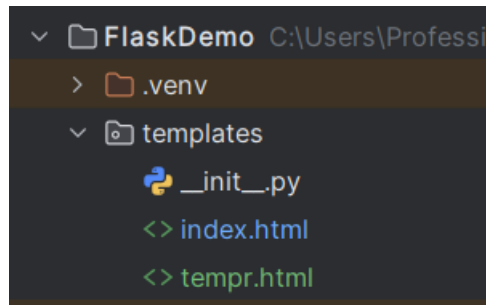


Рисунок 1 – Структура проекта

В html файле с названием index.html создайте футер с копирайтом. В копирайте укажите год и Ваше имя и фамилию.

В дальнейшем к страницам будут применены стили CSS. Также в домашнюю страницу может быть добавлено изображение при помощи тега src по Вашему желанию.

С целью соблюдения правил семантической верстки заголовков приложения и навигационное меню должны находиться внутри тега <header>, а само навигационное меню внутри тега <nav>. Основной контент должен быть реализован внутри тега <main>, а подвальная часть страницы с Вашим именем и знаком копирайта внутри тега <footer>.

Реализация backend приложения

Контроллер, состоящий из набора эндпоинтов реализуйте в файле с названием app.py.

Создайте первый эндпоинт, который при запуске приложения должен возвращать на фронт-энд шаблон созданной Вами домашней страницы приложения (рисунок 2)

```
from flask import Flask, request
from flask import render_template

app = Flask(__name__)

@app.route(rule: '/', methods=['GET']) new *
def index():
    return render_template('index.html')
```

Рисунок 2. – Создание эндпоинта домашней страницы

Предварительно импортируйте класс Flask, а затем создайте его экземпляр с названием app как показано на рисунке 3. Импортируйте request (он понадобится позже), а также render_template для отправки шаблонов. render_template принимает в качестве аргумента HTML файл который будет отправлен при переходе в данный эндпоинт.

Затем добавьте следующий код для запуска приложения в режиме отладки (рисунок 3).

```

from flask import Flask, request
from flask import render_template

app = Flask(__name__)

@app.route(rule: '/', methods=['GET']) new *
def index():
    return render_template('index.html')

if __name__ == '__main__':
    app.run(debug=True)

```

Рисунок 3 – Запуск приложения в режиме отладки

Осуществите запуск приложения путем выполнения команды в терминале:

```
python app.py
```

или команды:

```
flask run
```

В случае отсутствия ошибок должно появиться следующее сообщение в терминале:

```

PS C:\Users\Professional\Desktop\HackRF\FlaskDemo> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 883-211-756
127.0.0.1 - - [21/Sep/2025 08:57:31] "GET / HTTP/1.1" 200 -

```

Данное сообщение указывает, что веб-приложение запущено на локальном хосте.

При переходе по URL <http://127.0.0.1:5000> в браузере должна быть отражена страница веб-приложения, возвращаемая из соответствующего эндпоинта.

Остановка сервера осуществляется командой CTRL + C из терминала.

Для реализации задания потребуется модель, которая должна быть предварительно обучена (например, в «Google Colab»), а ее веса сохранены в специальном файле с расширением h5.

Для сохранения файла с весами, после обучения модели, в среде «Google Colab», где обучалась модель добавьте код:

```
model.save_weights('best.weights.h5')
```

Затем требуется сохранить веса где-то локально, чтобы затем их добавить в проект. Веса из «Google Colab» сохраняются как показано на рисунке

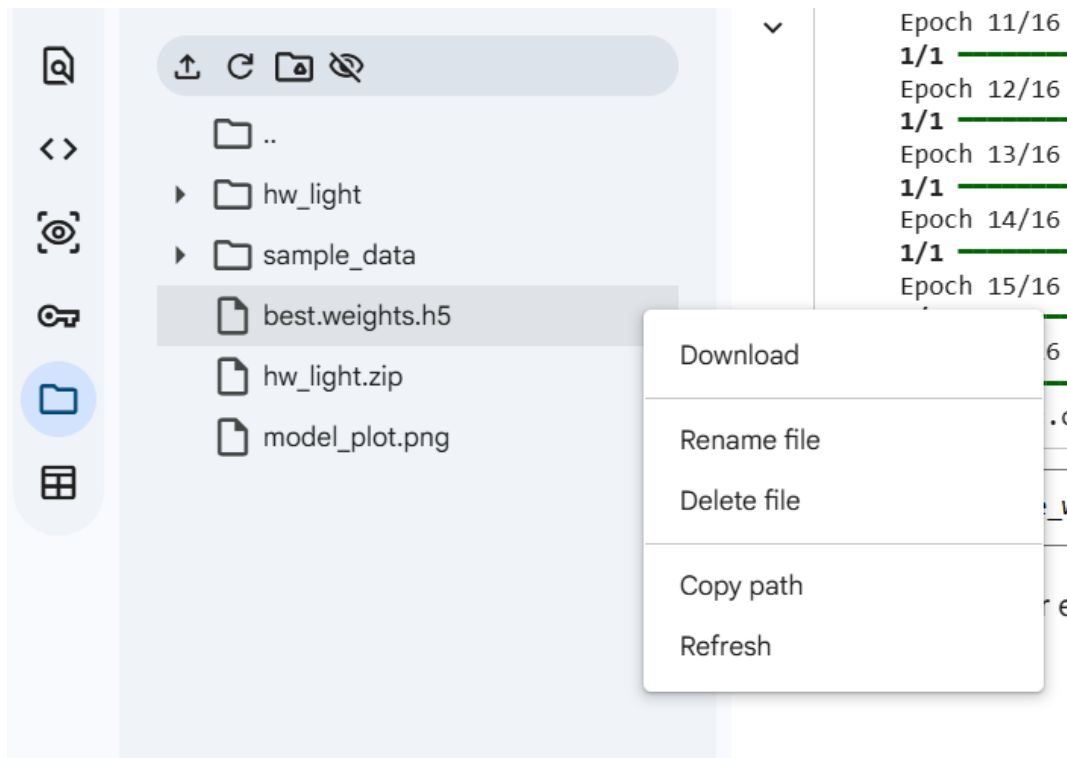


Рисунок 5. – Сохранение весов

Здесь нужно выбрать опцию «download» чтобы сохранить файл с весами на локальной машине.

Создайте в проекте пакет с названием «model» (рисунок 6). В нем будет находиться код архитектуры модели, который надо будет скопировать из среды, где модель была обучена и веса модели, в данном примере они находятся в файле с названием `'best.weights.h5'`.

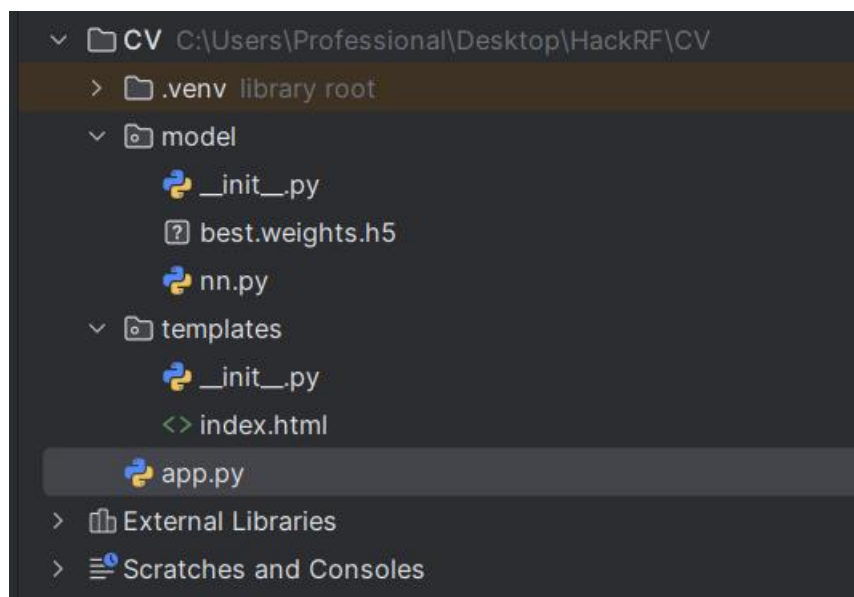
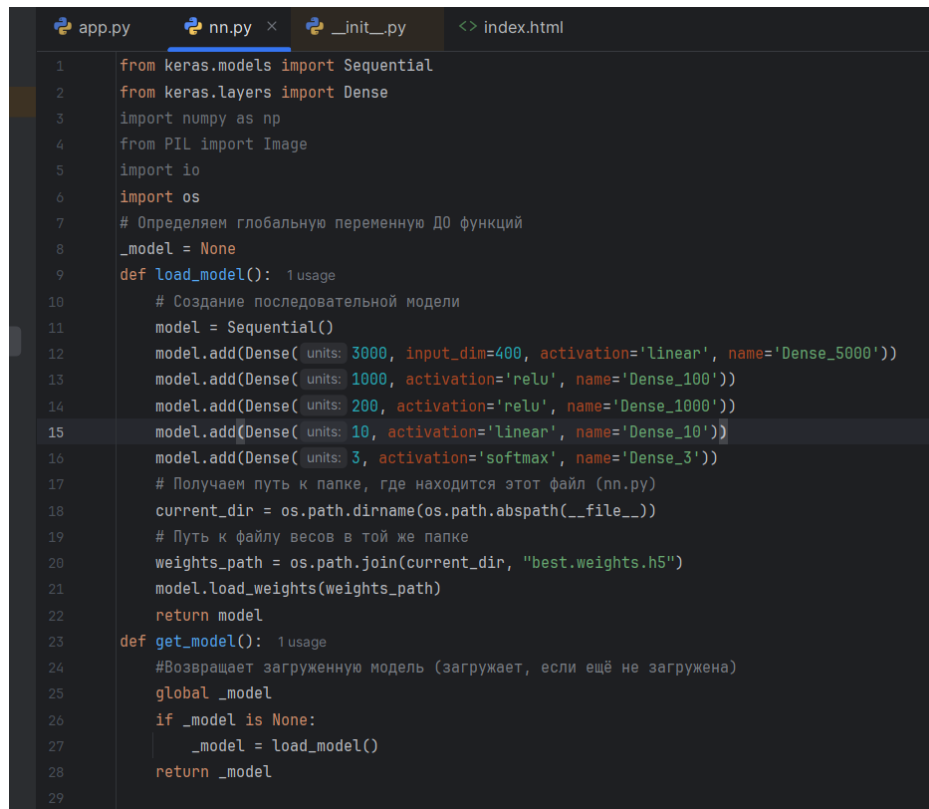


Рисунок 6 – Структура проекта

В файле, который назовите «nn.py» разместите архитектуру модели обернув в ее в функция (рисунок 7). Архитектура Вашей модели может быть отлична от той, которая приведена в примере на рисунке 7.

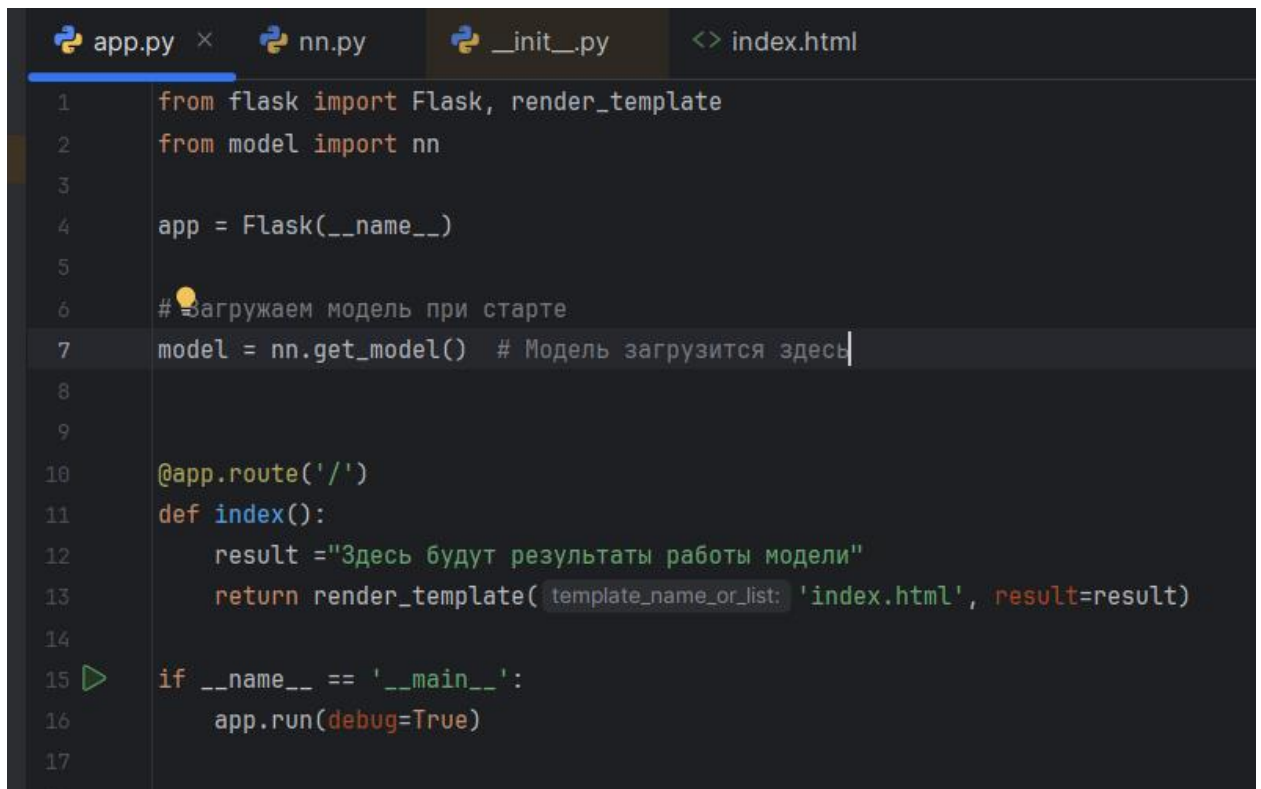


```
1 from keras.models import Sequential
2 from keras.layers import Dense
3 import numpy as np
4 from PIL import Image
5 import io
6 import os
7 # Определяем глобальную переменную Д0 функций
8 _model = None
9 def load_model(): 1 usage
10     # Создание последовательной модели
11     model = Sequential()
12     model.add(Dense(units=3000, input_dim=400, activation='linear', name='Dense_5000'))
13     model.add(Dense(units=1000, activation='relu', name='Dense_100'))
14     model.add(Dense(units=200, activation='relu', name='Dense_1000'))
15     model.add(Dense(units=10, activation='linear', name='Dense_10'))
16     model.add(Dense(units=3, activation='softmax', name='Dense_3'))
17     # Получаем путь к папке, где находится этот файл (nn.py)
18     current_dir = os.path.dirname(os.path.abspath(__file__))
19     # Путь к файлу весов в той же папке
20     weights_path = os.path.join(current_dir, "best.weights.h5")
21     model.load_weights(weights_path)
22     return model
23 def get_model(): 1 usage
24     #Возвращает загруженную модель (загружает, если ещё не загружена)
25     global _model
26     if _model is None:
27         _model = load_model()
28     return _model
29
```

Рисунок 7 – Архитектура модели

Важно не забыть сделать соответствующие импорты.

Реализуйте контроллер, находящийся в файле с названием app.py как показано на рисунке 8.



```
1 from flask import Flask, render_template
2 from model import nn
3
4 app = Flask(__name__)
5
6 # Загружаем модель при старте
7 model = nn.get_model() # Модель загрузится здесь
8
9
10 @app.route('/')
11 def index():
12     result = "Здесь будут результаты работы модели"
13     return render_template(template_name_or_list='index.html', result=result)
14
15 if __name__ == '__main__':
16     app.run(debug=True)
17
```

Рисунок 8 – Реализация контроллера веб-приложения

В контроллер надо импортировать модель, затем модель будет использована в соответствующем эндпоинте для классификации или предсказания. Результат предсказания будет находиться в переменной «result» и возвращен вместе с шаблоном на фронтэнд. На данном этапе в переменной «result» находится строка «Здесь будет результат работы модели». Поэтому, при запуске приложения, если все выполнено корректно, то в окне браузера появится html страница, приведенная на рисунке 9.



Здесь будут результаты работы модели

Рисунок 9 – Результат запуска приложения

В дальнейшем в приложение будет добавлена возможность загрузки изображения, код обработки изображения перед подачей в модель, и реализована задача классификации изображения с использованием обученной модели искусственной нейронной сети.

Перечень ссылок:

1. Документация Flask: <https://flask.palletsprojects.com/en/stable/>
2. Методические указания о начале работы и установке Flask: <https://rutube.ru/video/5d5132472bd7dca9e828d1b77cbba284>
3. Ссылка на ноутбук в Google Colab с моделью искусственной нейронной сети, использованной в веб-приложении: https://colab.research.google.com/drive/1uPrZ_rfAoNG57kaaxm30yE_D6amkATTn?usp=sharing